libp2p 통합

개요

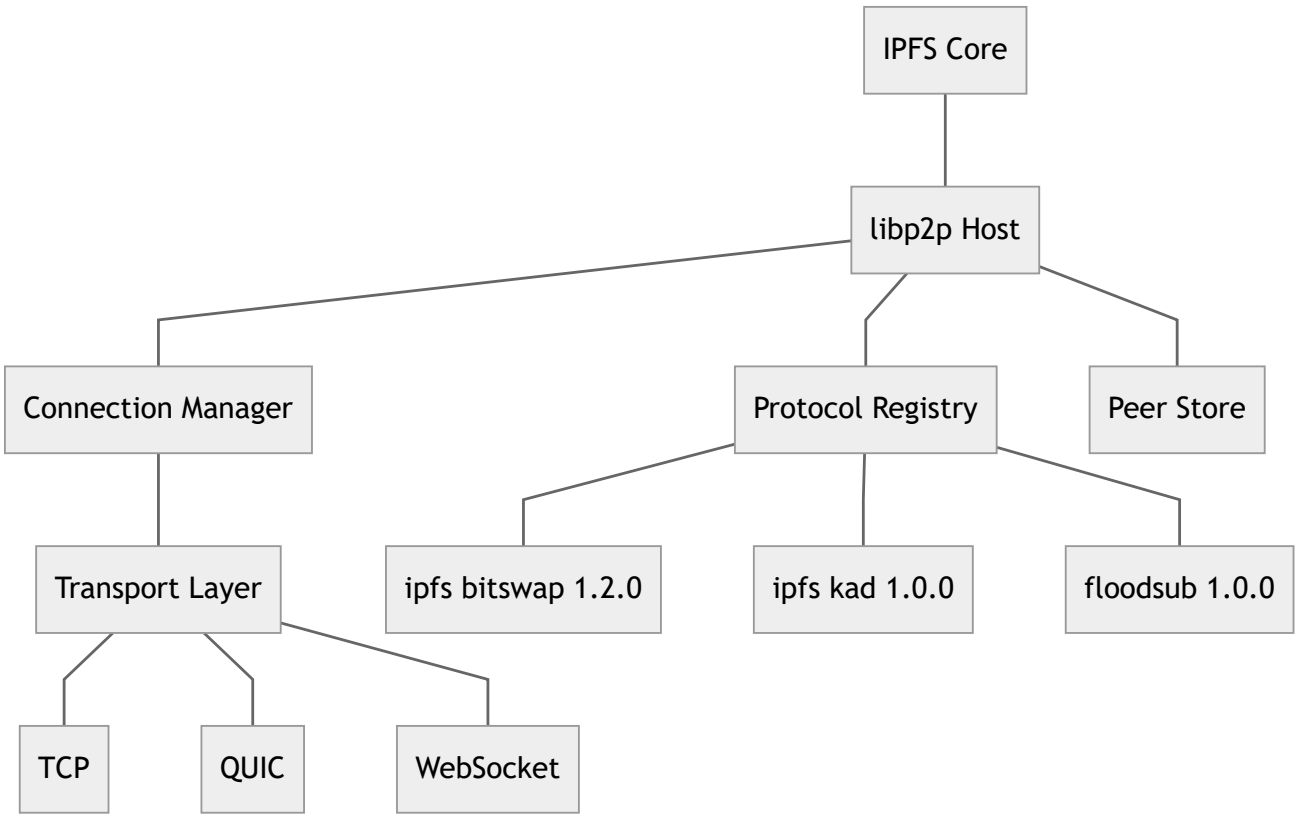
이 문서는 IPFS에서 libp2p를 통합하고 사용하는 방식을 상세히 설명합니다.

libp2p 아키텍처

모듈형 설계

- **Transport**: 네트워크 전송 계층
- **Security**: 암호화 및 인증
- **Stream Muxing**: 스트림 다중화
- **Peer Discovery**: 피어 발견
- **Routing**: 메시지 라우팅

IPFS 통합 구조



Host 초기화

기본 설정

```
func  createLibp2pHost(ctx context.Context, cfg *config.Config) (host.Host, erro

var  opts []libp2p.Option

// Identity 설정

priv, err := cfg.Identity.DecodePrivateKey("")

if  err != nil {

return  nil, err

}

opts = append(opts, libp2p.Identity(priv))
```

```
// Listen addresses 설정

opts = append(opts, libp2p.ListenAddrStrings(cfg.Addresses.Swarm...))

// Transport 설정

opts = append(opts, libp2p.Transport(tcp.NewTCPTransport))

opts = append(opts, libp2p.Transport(quic.NewTransport))

// Security 설정

opts = append(opts, libp2p.Security(tls.ID, tls.New))

opts = append(opts, libp2p.Security(noise.ID, noise.New))

// Muxer 설정

opts = append(opts, libp2p.Muxer(yamux.ID, yamux.DefaultTransport))

return  libp2p.New(opts...)

}
```

고급 설정 옵션

- **Connection Manager**: 연결 수 제한 및 관리
- **Resource Manager**: 리소스 사용량 제어
- **Relay**: 중계 노드 설정
- **AutoNAT**: NAT 상태 자동 감지

프로토콜 등록

프로토콜 핸들러

```
// Bitswap 프로토콜 등록
```

```
host.SetStreamHandler(bitswap.ProtocolBitswap, bitswapHandler)


// DHT 프로토콜 등록
host.SetStreamHandler(dht.ProtocolDHT, dhtHandler)


// Pubsub 프로토콜 등록
host.SetStreamHandler(pubsub.GossipSubID, pubsubHandler)
```

프로토콜 협상

1. **연결 수립**: 피어 간 기본 연결 생성
2. **프로토콜 제안**: 지원하는 프로토콜 목록 교환
3. **프로토콜 선택**: 공통 지원 프로토콜 선택
4. **스트림 생성**: 선택된 프로토콜로 스트림 생성

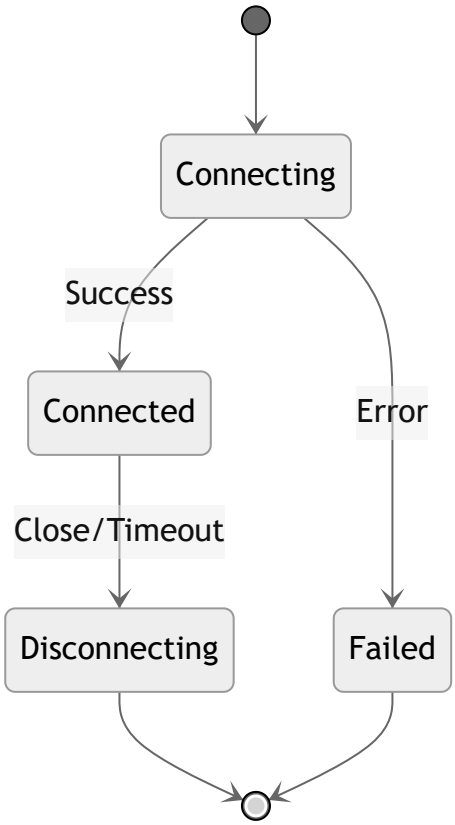
연결 관리

Connection Manager 설정

```
type ConnMgrConfig struct {
LowWater  int  // 최소 연결 수
HighWater  int  // 최대 연결 수
GracePeriod time.Duration // 연결 유지 기간
}
```

```
func setupConnMgr(cfg ConnMgrConfig) connmgr.ConnManager {
return connmgr.NewConnManager(
cfg.LowWater,
cfg.HighWater,
connmgr.WithGracePeriod(cfg.GracePeriod),
)
}
```

연결 생명주기



보안 계층

암호화 프로토콜

- **TLS 1.3**: 표준 TLS 암호화
- **Noise**: 경량 암호화 프로토콜
- **선택 기준**: 성능 vs 호환성

인증 과정

1. **키 교환**: 공개키 기반 키 교환
2. **신원 확인**: Peer ID 검증
3. **세션 키 생성**: 대칭키 생성
4. **보안 채널 수립**: 암호화된 통신 채널

스트림 다중화

Yamux 멀티플렉서

- **특징**: Go 네이티브 구현
- **성능**: 낮은 오버헤드
- **기능**: 흐름 제어, 백프레셔

스트림 관리

```
// 새 스트림 생성
stream, err := host.NewStream(ctx, peerID, protocol.ID("/ipfs/bitswap/1.2.0"))

// 스트림 읽기/쓰기
writer := bufio.NewWriter(stream)
reader := bufio.NewReader(stream)
```

```
// 스트림 종료
stream.Close()
```

피어 저장소

Peer Store 구조

```
type PeerStore interface {
PeerInfo(peer.ID) peer.AddrInfo
Peers() peer.IDSlice
Addrs(peer.ID) []ma.Multiaddr
AddAddrs(peer.ID, []ma.Multiaddr, time.Duration)
SetAddrs(peer.ID, []ma.Multiaddr, time.Duration)
UpdateAddrs(peer.ID, time.Duration, time.Duration)
}
```

메타데이터 관리

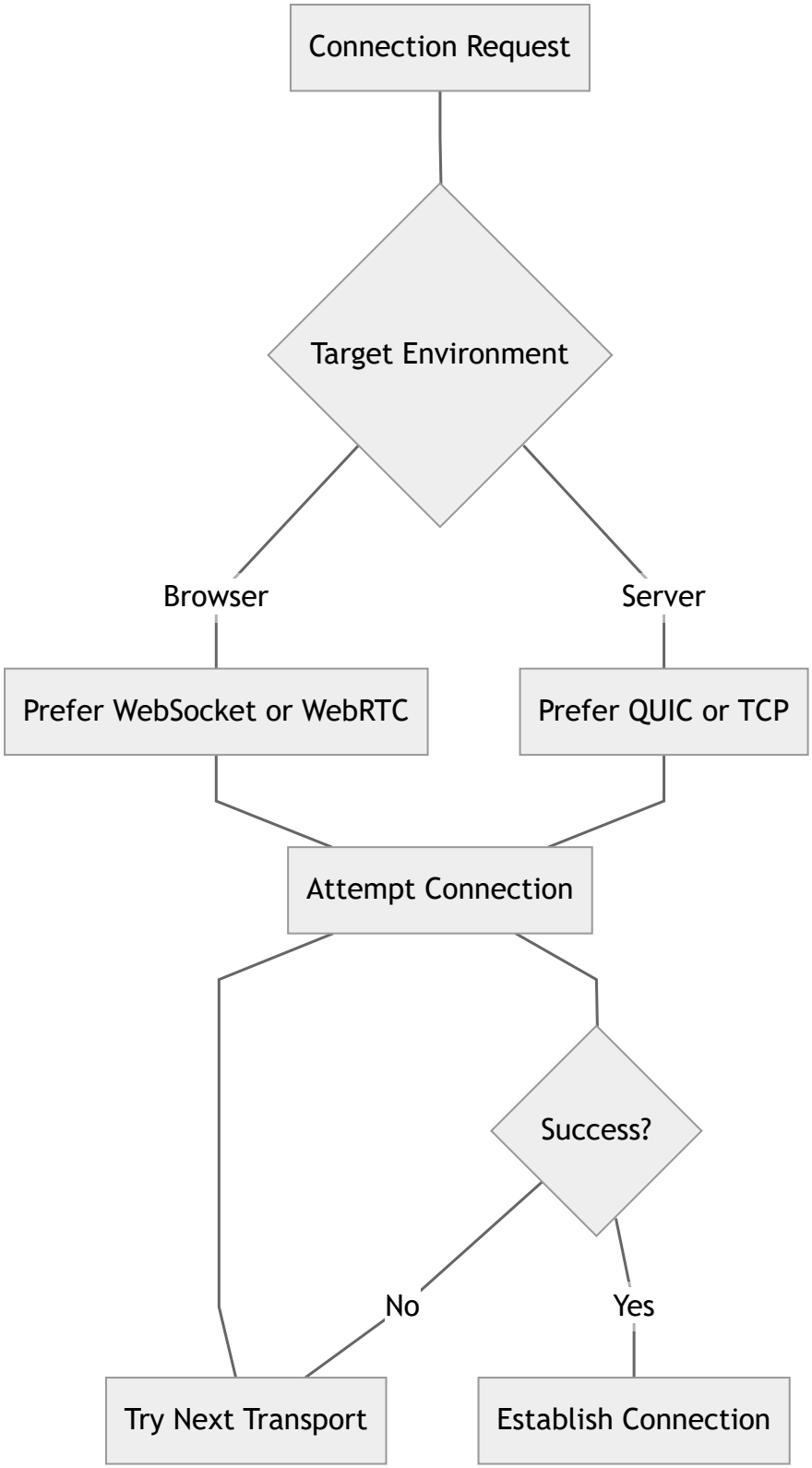
- **주소 정보**: 피어의 네트워크 주소
- **프로토콜 지원**: 지원하는 프로토콜 목록
- **연결 기록**: 과거 연결 이력
- **성능 메트릭**: 지연시간, 대역폭 등

네트워크 전송

지원 전송 프로토콜

- **TCP**: 신뢰성 있는 연결 지향 프로토콜
- **QUIC**: UDP 기반 고성능 프로토콜
- **WebSocket**: 웹 브라우저 호환성
- **WebRTC**: 브라우저 간 직접 연결

전송 선택 전략



성능 최적화

연결 풀링

- **재사용**: 기존 연결 재활용

- **사전 연결**: 예상 피어에 미리 연결
- **연결 워밍**: 유휴 연결 활성 상태 유지

리소스 관리

```
type ResourceManagerConfig struct {
MaxMemory int64
MaxFD int
MaxConns int
MaxConnsPerIP int
MaxStreams int
}
```

모니터링 및 디버깅

메트릭 수집

- **연결 통계**: 활성 연결 수, 연결 성공률
- **대역폭 사용량**: 송신/수신 바이트
- **프로토콜 사용량**: 프로토콜별 트래픽

디버깅 도구

```
# libp2p 연결 상태
ipfs swarm peers
```

```
# 피어 정보 확인
ipfs id <peer-id>

# 네트워크 진단
ipfs diag net

# 로그 레벨 설정
ipfs log level swarm debug
```

참고 자료

- P2P 기본 원리
- 피어 발견 메커니즘
- Bitswap 프로토콜